

SGExplainer: Balanced Path-based Signed Graph Neural Network Explanation for Link Sign Prediction

Jie Gao
University of Exeter
Exeter, UK
jg911@exeter.ac.uk

Geyong Min*
University of Exeter
Exeter, UK
g.min@exeter.ac.uk

Jia Hu*
University of Exeter
Exeter, UK
j.hu@exeter.ac.uk

Fei Hao
Shaanxi Normal University
Xi'an, China
fhao@snnu.edu.cn

Abstract

Signed Graph Neural Networks (SGNNs) have achieved outstanding performance in Link Sign Prediction (LSP), which involves predicting the existence and polarity of edges, by effectively modeling positive and negative interactions in signed graphs. However, their black-box nature raises transparency concerns, necessitating faithful explanations of model behavior to ensure trustworthiness and accountability. Existing eXplainable GNN (XGNN) methods, primarily designed for unsigned graphs, struggle to provide meaningful and human-understandable explanations for SGNN-based LSP, often generating disconnected subgraph explanations or neglecting the unique sign interactions. To address the gap, we propose SGExplainer, a novel method that leverages balanced paths, a concept rooted in signed graph theory, to provide clear and faithful explanations for LSP. SGExplainer employs a path-enforcing mask learning framework that ensures interpretable balanced path generation while maintaining explanation fidelity. Extensive experiments on real-world signed graphs demonstrate that SGExplainer consistently provides faithful and intuitive explanations for various SGNNs, outperforming state-of-the-art baselines in explanation quality, interpretability, and efficiency.

CCS Concepts

• Computing methodologies → Neural networks; • Mathematics of computing → Graph algorithms.

Keywords

Signed Graph, Graph Neural Network, Model Explanation

ACM Reference Format:

Jie Gao, Jia Hu, Geyong Min, and Fei Hao. 2026. SGExplainer: Balanced Path-based Signed Graph Neural Network Explanation for Link Sign Prediction. In *Proceedings of the ACM Web Conference 2026 (WWW '26)*, April 13–17, 2026, Dubai, United Arab Emirates. ACM, New York, NY, USA, 12 pages. <https://doi.org/10.1145/3774904.3792146>

*Corresponding Authors.



This work is licensed under a Creative Commons Attribution 4.0 International License. *WWW '26, Dubai, United Arab Emirates*
© 2026 Copyright held by the owner/author(s).
ACM ISBN 979-8-4007-2307-0/2026/04
<https://doi.org/10.1145/3774904.3792146>

1 Introduction

In many complex real-world systems, such as social networks, economic networks and biological networks, interactions between entities extend beyond conventional positive relations like friendship, trust and agreement, to also encompass negative ties such as enmity, distrust and conflict [37]. These polarized interactions can be modeled as a *signed graph* [2, 6, 28, 38], where each edge is annotated with either a positive or negative sign. As illustrated in Fig. 1(a)(b), users in social networks form friendships or express hostility, while users in review platforms support or oppose each other's opinions. To capture such complex signed relationships, *Signed Graph Neural Networks* (SGNNs) [8, 18, 19] have emerged as powerful tools for learning node representations and achieved state-of-the-art performance in *Link Sign Prediction* (LSP) [16, 24]. *The LSP task, which involves inferring not only the presence of unobserved or future edges but also their signs*, is the central problem in signed graph analysis and has proven indispensable across various applications, such as social recommendation [26], trust assessment [13] and biological network analysis [12].

However, despite their widespread success, SGNNs often operate as black-box models, raising significant concerns about their transparency and accountability [5]. As illustrated in Fig. 1(a), it is still unclear (*Q1*) *why the model recommends a potential friendship between two users*. This lack of interpretability becomes especially pressing in high-stake fields such as finance [29] and healthcare [25], where the inability to trace the reasoning behind predictions hinders efforts to detect biases or ensure fairness in outcomes [15]. As a result, research in explainable artificial intelligence (XAI) [9, 17] has attracted growing attention, aiming at providing transparent rationales for model predictions. Such explanations not only help stakeholders to understand why certain predictions are made but also enable verification that model decisions align with domain-specific knowledge [4, 23].

In response to this need, various explainable GNN (XGNN) methods [21, 30–33] have been proposed to identify which subgraphs are most critical in determining model predictions for node- and graph-level tasks. However, to the best of our knowledge, no post-hoc explanation methods have been developed specifically for SGNNs. Moreover, the subgraph explanations generated by these methods face significant limitations when generalized to explaining LSP: 1) *Connection interpretability*. Since LSP involves a source-target node

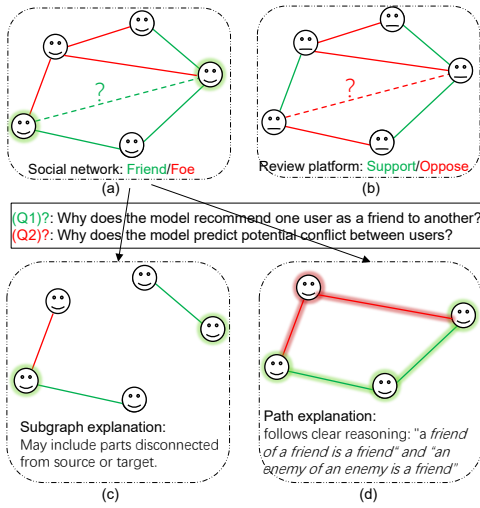


Figure 1: Illustration of signed graphs and comparison of subgraph-based vs. balanced-path-based explanations. (a) A social network with friend/foe links. (b) A review platform with support/oppose relations. Green/red lines denote positive/negative edges; dashed lines are predicted edges. To answer question Q1: *why does the model recommend one user as a friend to another*, (c) subgraph-based explanations include disconnected components between the source and target nodes, making it less convincing. (d) In contrast, balanced-path-based explanations offer more intuitive reasoning, such as “a friend of a friend is a friend” and “an enemy of an enemy is a friend”.

pair, a convincing explanation should highlight the essential connections between them, as these connections reveal the information flow that underlies and justifies the predicted link [35]. However, existing methods [21, 30, 33] often produce disconnected subgraphs as explanations (e.g., Fig. 1(c)), which fail to capture meaningful connections and may instead introduce noisy or irrelevant structures, making the explanations difficult for humans to interpret and trust. 2) *Sign interaction*. An effective explanation for LSP should reveal the unique sign interactions that influence model prediction, i.e., how positive and negative edges interact to produce the predicted link sign [37]. However, existing XGNN methods [21, 30–33, 35] only work for unsigned graphs; therefore, when generalized to explaining LSP, their provided explanations (e.g., Fig. 1(c)) overlook these crucial unique sign interactions and fail to align with the domain knowledge inherent in signed graphs.

These limitations highlight that, in addition to being faithful to the model’s predictions, explanations for LSP must also be connection-aware and sign-sensitive to be understandable and trustworthy. To meet these requirements, we draw on the concept of *balanced paths* (see Section 3), which serves as a principled way to model sign-aware connections between nodes. Rooted in the fundamental *structural balance theory* [7, 10], balanced paths encode how positive and negative edges combine along a path, thereby capturing the underlying sign interactions that drive link sign predictions. For example, as shown in Fig. 1(d), balanced-path-based explanations

provide a clear reasoning chain that answers the motivating question Q1: *a friend of a friend tends to be a friend, and an enemy of an enemy tends to be a friend*. By capturing these intuitive sign-aware connections between source-target nodes, balanced paths naturally reflect their local and global structural proximity [22], making them a suitable foundation for explaining LSP.

Based on the observations above, we propose SGExplainer, a novel post-hoc explainable method that generate clear and intuitive balanced-path-based explanations for SGNN-based LSP, effectively addressing the limitations posed by existing methods. Explaining LSP with balanced paths presents two significant challenges: (a) ensuring explanation fidelity while constraining explanations to interpretable balanced paths, and (b) ensuring efficiency in practical applications that demand explanations for numerous instances. To tackle these challenges, we adopt the mask learning paradigm and devise a balanced-path-enforcing learning framework to promote the generation of fidelity-preserving balanced paths. Specifically, we design a Multi-Layer Perceptron (MLP)-based mask generator that, once trained, can efficiently explain unseen instances in an inductive manner. To encourage high-fidelity balanced path generation, we introduce two complementary losses: a mutual-information-based prediction loss to maintain faithfulness and a balanced-path-enforcing loss to promote explanations in the form of balanced paths. In addition, we decouple path search from mask training by designing an efficient balanced-path search algorithm that is executed once before training, thereby avoiding costly repeated searches and guiding the strengthening of all on-path edges during mask learning. Our main contributions are summarized as follows:

- To the best of our knowledge, this is the first work to provide post-hoc explanations for SGNN models. SGExplainer functions as a plug-and-play method, compatible with various SGNN models, and searches for concise balanced paths that serve as clear and human-understandable explanations.
- We design a balanced-path-enforcing mask learning framework that integrates an inductive mask generator, tailored loss functions, and a decoupled balanced-path search algorithm, ensuring the generated explanations are faithful, interpretable, and efficient.
- Extensive experiments demonstrate that SGExplainer generates superior explanations for representative SGNN models, outperforming state-of-the-art explainers in terms of explanation quality, interpretability, and efficiency.

2 Related Work

Signed Graph Neural Networks. In recent years, Signed Graph Neural Networks (SGNNs) [3, 8, 11, 14, 18–20, 27, 36] have emerged as state-of-the-art methods for learning on signed graphs, with Link Sign Prediction (LSP) being the most widely studied and closely associated task due to the coexistence of positive and negative edges. The SGCN [8], one of the earliest and important SGNN models designed for LSP, aggregates messages from positive and negative neighbors separately and propagates them in a layered scheme inspired by *balance theory*. Following this, SNEA [18] introduces a sign-aware attention mechanism, allowing the model to learn

the relative importance of neighbors with different sign. LightS-GCN [19] builds upon these insights by proposing a lightweight architecture that retains the sign separation principle but removes nonlinearities and collapses layers for computational efficiency. Beyond these models, recent SGNNs have extended beyond designing sign aggregation to introducing contrastive (e.g., SGCL [27]) and adversarial objectives (e.g., ASiNE[14], SigGAN [3]). Despite varying designs, the majority of SGNNs maintain a strong alignment with the LSP task, i.e., optimizing their architectures or training procedures to learn higher-quality node embeddings to infer edge polarity. For a comprehensive overview of this evolving field, please refer to the recent survey [37].

Table 1: Comparison of explanation methods in terms of desired properties.

Methods	GNNExplainer [30]	PGExplainer [21]	SubgraphX [33]	PaGE-Link [35]	SE-SGformer [16]	SGExplainer
Explains SGNN	×	×	×	×	×	✓
Explains LSP	?	?	?	?	✓	✓
Path-based	×	×	×	✓	×	✓
Scalability	×	✓	×	×	?	✓
Post-hoc	✓	✓	✓	✓	×	✓

GNN Explanations. Table 1 summarize the properties of our SGExplainer vs. representative explanation methods. GNN explanation methods aim to reveal model decision-making processes by identifying influential substructures or features, differing mainly in their definition of importance and subgraph selections. One of the foundational methods is GNNExplainer [30], which learns a compact subgraph and feature subset by optimizing edge and feature masks to maximize mutual information with the original prediction. Building on this, PGExplainer [21] introduces a probabilistic edge-masking mechanism that can generalize across unseen instances, improving scalability and training efficiency. Other methods, such as SubgraphX [33] and GStarX [34], adopt game-theoretic values to assign importance scores to nodes, edges and subgraphs based on their contribution to the final prediction. For a broader overview of GNN explanation methods, please refer to recent surveys [1, 32].

Despite these advances, most explanation methods have been developed for node classification or graph classification tasks. Extending explainability to link prediction, our work is most closely related to PaGE-Link [35] and SE-SGformer [16]. PaGE-Link is designed to generate path-based explanation for heterogeneous link prediction. However, PaGE-Link requires retraining for each instance, limiting its scalability significantly when a large number of instances to be explained in real applications. In contrast, our SGExplainer employs an inductive mask learning paradigm, enabling efficient explanations for unseen instances without per-instance optimization. Second, it focuses on heterogeneous link prediction, whereas we address the new downstream task of link sign prediction, which involves distinct challenges arising from signed edge interactions. Another closely related work SE-SGformer [16] is a recent transformer-based method for signed graphs that explains predictions using K-nearest neighbors. However, this heuristic neighbor-based mechanism neither guarantees faithful model

reasoning nor overcomes the aforementioned limitations inherent to subgraph-based explanations. In addition, its transformer architecture differs fundamentally from message-passing SGNNs. By contrast, SGExplainer introduces a post-hoc balanced-path-enforcing framework for SGNN-based LSP, producing explanations that are faithful to model reasoning and aligned with domain knowledge of signed graphs. Since the implementation of SE-SGformer is not publicly available, we omit it from empirical comparison.

3 Preliminaries and Problem Formulation

DEFINITION 1 (SIGNED GRAPH). A signed graph is a simple graph $\mathcal{G} = (\mathcal{V}, \mathcal{E}^+, \mathcal{E}^-)$, where $\mathcal{V}$ is the set of nodes, and $\mathcal{E}^+$ and $\mathcal{E}^-$ are the sets of positive and negative edges, respectively. Each edge $e \in \mathcal{E}^+$ is labeled ‘+1’ (or ‘+’), and each $e \in \mathcal{E}^-$ is labeled ‘-1’ (or ‘-’).

DEFINITION 2 (BALANCED PATH). In a signed graph $\mathcal{G}$, the sign of a path between nodes s and t is computed as the product of the signs of on-path edges: the path is positive if it contains an even number of negative edges, and negative otherwise. A balanced path is defined as a simple path from s and t that has the same sign as the edge (s, t) .

3.1 SGNN-based Link Sign Prediction

Link Sign Prediction (LSP) is a fundamental task in signed graph analysis, which requires *not only to predict the presence of missing edges but also to determine its sign*. In recent years, SGNNs [8, 11, 18–20] have emerged as the dominant approach for addressing LSP task. These models typically adopt an encoder-decoder architecture that jointly learns node representations and predicts edge signs. The encoder Φ_e of an SGNN model learns node embeddings through carefully designed message passing and aggregation schemes that are tailored to signed graphs. The decoder Φ_d , typically implemented as a multinomial logistic regression model, takes the embeddings and outputs the probabilities that an edge between nodes s and t is positive, negative, or absent. Formally, let $\Phi(\cdot, \cdot)$ denote a well-trained SGNN model for predicting the unseen signed links in $\mathcal{G}$ with encoder Φ_e and decoder Φ_d . The SGNN model can be represented as:

$$Z = \Phi_e(\mathcal{G}); \quad Y = \arg \max \Phi_d(z_s, z_t). \quad (1)$$

where Φ_e learns node embeddings Z , and then Φ_d takes node embeddings and outputs the predicted probability across the edge classes, and Y denotes the predicted class.

3.2 Balance Theory-based Link Sign Prediction

The traditional theoretical foundation for LSP is *structural balance theory* [7, 10], a social-psychological principle stating that signed

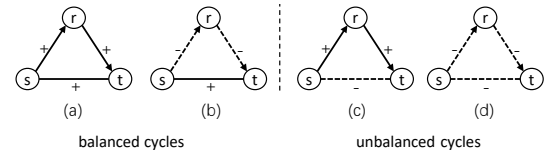


Figure 2: Illustration of balanced and unbalanced cycle. An unbalanced (resp. unbalanced) cycle is defined as containing an odd (resp. even) number of negative edges.

graphs tend to evolve toward self-consistent and to stable balanced configurations. The notion of *balance* is typically defined over *triads/cycles*: a signed graph is balanced if every cycle contains an even number of negative edges (see Fig. 2). Based on this principle, the missing link between nodes s and t can be inferred from existing paths: a path $s - r - t$ with an even (resp. odd) number of negative edges implies a positive (resp. negative) edge between s and t . The decisive signed path, which agrees with the predicted sign edge, is referred to as a *balanced path*. For example, in Fig. 2(b), the path $s - r - t$ has two negative edges, then a positive edge (s, t) is predicted to complete a closed balanced cycle, and $s - r - t$ is a balanced path. This formulation aligns with intuitive social rules such as “the friend of my friend is my friend” and “the enemy of my enemy is my friend”.

In a nutshell, balanced paths capture structural proximity between nodes by encoding local and global sign interactions, thus determining link signs while offering transparent and human-intuitive justification for predictions. Therefore, we embrace balanced paths as the explanatory structure for SGNN-based LSP.

3.3 Problem Statement

In this paper, we focus on a *post-hoc* and *instance-level* SGNN-based LSP explanation problem. The post-hoc refers to the fact that the SGNN model $\Phi(\cdot, \cdot)$ has been well-trained, and the explanation method won’t alter its architecture or parameters. The instance level means that, for each instance (s, t) , the explanation method aims to provide a rationale for why the unseen signed edge (s, t) is predicted by the pre-trained model $\Phi(\cdot, \cdot)$. In a practical use case, such as social recommendation, the explanation could answer questions like: *why does a user trust or distrust another user?*

A $\mathcal{L}$ -layer SGNN $\Phi(\cdot, \cdot)$ collects messages only from the $\mathcal{L}$ -hop neighbors of s and t to compute node embeddings z_s and z_t . So, the prediction $\Phi(\mathcal{G}, (s, t))$ is fully determined by the $\mathcal{L}$ -hop ego-graph $\mathcal{G}_c$ of the predicted node pair s and t , i.e., $\Phi(\mathcal{G}, (s, t)) \equiv \Phi(\mathcal{G}_c, (s, t))$. Therefore, to explain $\mathcal{L}$ -layer SGNN $\Phi(\mathcal{G}, (s, t))$, we restrict the search space to the *computation graph* $\mathcal{G}_c = (\mathcal{V}_c, \mathcal{E}_c^+, \mathcal{E}_c^-)$, where $\mathcal{G}_c$ is the induced $\mathcal{L}$ -hop ego-graph with node set $\mathcal{V}_c = \{v \in \mathcal{V} \mid \text{dist}(v, s) \leq \mathcal{L} \vee \text{dist}(v, t) \leq \mathcal{L}\}$.

As aforementioned, we adopt balanced paths as the explanatory structure for LSP explanation. Therefore, *the explanation for a SGNN prediction should highlight the balanced paths in $\mathcal{G}$ that are most influential to the prediction, ensuring both informativeness and fidelity while constraining the rationale to balanced paths*. Moreover, these paths should be concise, bounded by a maximum path length $l_{\max}$, since overly long paths tend to be harder to understand. In addition, we introduce a budget B to limit the maximum number of edges included in the explanation, enabling fair comparison across different methods. With these considerations, we formally define the explanation problem for SGNN-based LSP as follows:

PROBLEM 1 (BALANCED PATH-BASED LSP EXPLANATION). *Given a well-trained SGNN model $\Phi(\cdot, \cdot)$, a computation graph $\mathcal{G}_c$ of (s, t) induced from $\mathcal{G}$, a maximum path length $l_{\max}$, and an edge budget B , for link sign prediction $Y = \Phi(\mathcal{G}_c, (s, t))$, we aim to generate balanced path-based explanations $\mathcal{P} = \{p \mid p \text{ is an } s\text{-}t \text{ balanced path with } |p| \leq l_{\max}\}$, such that $|\mathcal{P}| \cdot l_{\max} \leq B$, while ensuring the explanations are concise and informative for the prediction.*

4 The SGExplainer

Overview. Fig. 3 illustrates the framework of SGExplainer, which consists of three modules: mask generator, mask learning, and path generation. The mask generator takes the computational graph and the node embeddings computed by a well-trained SGNN model as inputs, and produces an edge mask M that indicates the importance of each edge for the prediction. The mask learning module updates the mask generator using the designed balanced-path-aware training objective, encouraging higher weights on critical on-path edges and lower weights on irrelevant ones. Finally, the path generation module utilizes the generated edge mask M to select the shortest balanced paths, adhering to the explanation budget B , and produces the final explanations.

4.1 Mask Generator

SGExplainer learns a mask generator for explaining link sign prediction. We use a Multi-Layer Perceptron (MLP) as the mask generator, which takes node embeddings z_s, z_t as inputs to generate an edge mask M . Each element $M_{ij} \in M$ indicate the importance of edge (v_i, v_j) in $\mathcal{G}_c$ in contributing to model LSP Y . Specifically, we first obtain the node embeddings from the SGNN encoder Φ_e . Then, we concatenate these embeddings and pass them through the mask generator to compute edge mask M :

$$M = \Psi([z_s; z_t]); \quad z_s, z_t = \Phi_e(\mathcal{G}_c, (s, t)) \quad (2)$$

where Ψ is a parameterized MLP, $[;]$ denotes concatenation operation, and $M \in \mathbb{R}^{N \times N}$.

The generated M is applied to computational graph, i.e., $\mathcal{E}_c \odot \sigma(M)$, to obtain a *masked graph* $\mathcal{G}_m$, where $\odot$ is the element-wise product, and $\sigma(\cdot)$ is the sigmoid function. Feeding the masked graph into SGNN models will change the edge weights in message passing, allowing SGNNs to aggregate more information along highly weighted edges while reducing the influence of less important edges. By optimizing the mask generator, the computed edge mask M highlights the edges most influential to SGNN predictions. Once trained, the generator can explain unseen samples in an inductive manner.

4.2 Mask Learning

The general idea of mask learning is to optimize the mask generator to learn an edge mask M that assigns high weights to edges important for the prediction and low weights to less relevant ones. As discussed in Section 3.3, *for explaining the LSP task, explanatory edges should not only be influential for the model prediction but also form meaningful balanced paths*. To achieve these two objectives, we introduce two loss terms, prediction loss $\mathcal{L}_{\text{pred}}$ and path-enforcing loss $\mathcal{L}_{\text{path}}$. The former ensures that the masked graph retains fidelity to the original model prediction, while the latter encourages the learned explanation to form balanced path structures. The overall loss function is defined as:

$$\mathcal{L} = (1 - \lambda)\mathcal{L}_{\text{pred}} + \lambda\mathcal{L}_{\text{path}} \quad (3)$$

We combine $\mathcal{L}_{\text{pred}}$ and $\mathcal{L}_{\text{path}}$ using a weighted hyperparameter λ , enabling the explainer to produce balanced-path-based explanations that preserve both predictive accuracy and human-understandable structure. Next, we detail the prediction loss $\mathcal{L}_{\text{pred}}$ and the path-enforcing loss $\mathcal{L}_{\text{path}}$.

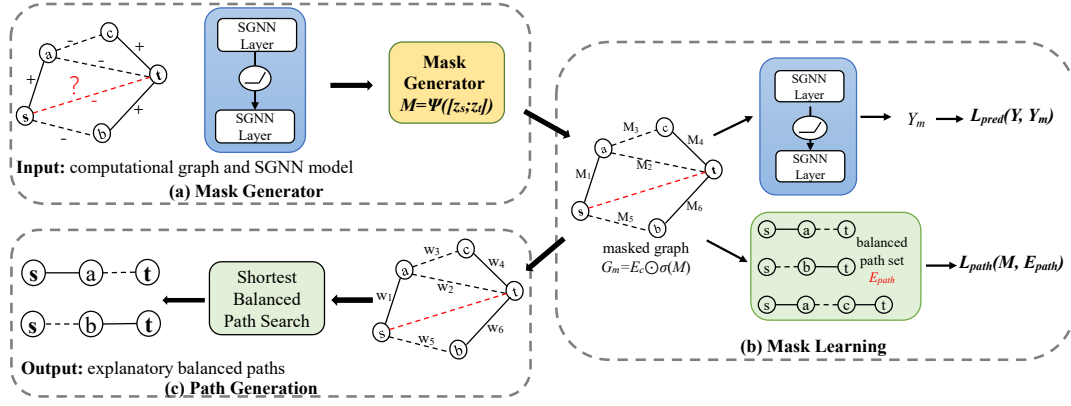


Figure 3: The overall framework of SGExplainer. Given a well-trained SGNN model and its predicted link (s, t) , SGExplainer output explanatory balanced paths to explain why the model predicts a negative link (s, t) . Solid lines represent positive edges, dashed lines represent negative edges.

4.2.1 The prediction loss. The $\mathcal{L}_{pred}$ guides the mask generator to identify important explanatory edges for model prediction, meaning the masked graph should retain the same predictive capability as the original graph. Following perturbation-based methods [21, 30, 35], an edge (v_i, v_j) is considered important if removing it significantly reduces the model confidence in the target prediction for (s, t) . The absence of such an edge serves as a strong counterfactual, suggesting that it should be retained in $\mathcal{G}_m$ with a higher weight. A well-masked graph $\mathcal{G}_m$ should preserve sufficient information to predict the target link (s, t) , similar to the full computation graph $\mathcal{G}_c$. This intuition can be formalized as maximizing the mutual information between the masked graph prediction and the original graph prediction, which is equivalent to minimize the prediction loss when the computation graph is limited to masked graph $\mathcal{G}_m$:

$$\mathcal{L}_{pred} = - \sum_{c=1}^C P_{\Phi}(Y = c \mid \mathcal{G}_c, (s, t)) \log P_{\Phi}(Y = c \mid \mathcal{G}_m, (s, t)) \quad (4)$$

s.t. $\mathcal{G}_m = (\mathcal{V}_c, \mathcal{E}_c \odot \sigma(M))$

where C is the number of labels, and $\mathcal{G}_m$ is the masked graph. By optimizing $\mathcal{L}_{pred}$, the mask generator encourages higher weights for important edges in the masked graph, thereby preserving their influence on the model's prediction.

4.2.2 The path-enforcing loss. The $\mathcal{L}_{path}$ guides the mask generator to highlight edges that form balanced paths and reinforce their significance in balanced path formation. The intuition behind is to identify candidate edges with the potential to form concise and informative balanced paths, and then encourage mask generator to increase their weights. Specifically, the path-enforcing loss can be formulated as:

$$\mathcal{L}_{path} = - \log \left(\frac{1}{|\mathcal{E}_{path}|} \sum_{e \in \mathcal{E}_{path}} \sigma(M_e) \right) \quad (5)$$

where $\mathcal{E}_{path}$ denotes the set of edges that form balanced paths between (s, t) with a maximum path length twice the number of SGNN layers, and $\sigma(M_e)$ is the sigmoid-transformed importance

score of edge e . The averaged score reflects the strength or importance of these paths, with higher values indicating more informative ones. By optimizing $\mathcal{L}_{path}$, the mask generator is encouraged to assign greater weights to path-forming edges, thereby promoting the generation of coherent and trustworthy balanced-path-based explanations.

4.2.3 Balanced path search. The key question for computing $\mathcal{L}_{path}$ is identifying $\mathcal{E}_{path}$, which contains all edges forming balanced paths between source-target pair (s, t) . To solve this, we propose a Shortest Balanced Path Search (SBPSearch) algorithm designed to find balanced paths between (s, t) with length up to $2\mathcal{L}$, where $\mathcal{L}$ denotes the number of layers in the SGNN model. The pseudo code can be found in Algorithm 2 in Appendix A.3. Considering that the identified paths only provide graph topology information for edge mask training, we safely decouple path search from mask training. The balanced paths are identified prior to training, and then used to guide mask learning process. This ensures that path identification is performed independently of the initial edge mask, avoiding the introduction of early-stage noise. In contrast, PaGE-Link [35] performs shortest path searches at every training epoch, incurring substantial computational cost. Moreover, these searches are guided by a randomly initialized and underfitted mask, causing early-selected path edges to be repeatedly reinforced in subsequent epochs. In summary, our method significantly reduces computational complexity while ensuring stable and meaningful path selection throughout training.

4.3 Path Generation and Complexity Analysis

After the mask learning converges, we use the trained explainer to compute the edge mask M for each instance via Eq. (2). The obtained edge mask M encodes the explanatory importance of each edge to the target instance. Then, we run SBPSearch algorithm on the inverse values of M to identify shortest balanced paths as explanations, adhering to maximum path length l_{max} and edge budget B .

$$\mathcal{P} \leftarrow \text{SBPSearch}(\mathcal{G}_c, s, t, l_{max}, M, B) \quad (6)$$

4.3.1 Complexity Analysis. The pseudo-code of SGExplainer is presented in Algorithm 1. Next, we analyze its time complexity. A detailed analysis can be found in Appendix A.4.

PROPOSITION 1. *Let I_t denote the number of training instances, where each instance has a computation graph $\mathcal{G}_c = (\mathcal{V}_c, \mathcal{E}_c)$, and T be the number of learning epochs, the training complexity of SGExplainer is $O(I_t(\Delta_{\text{path}} + T|\mathcal{E}_c|))$.*

SGExplainer decouples balanced path search from mask learning. The SBPSearch is executed once before training. Its time complexity is analyzed in Appendix A.3 and denoted as $\Delta_{\text{path}} = O((|\mathcal{V}_c| + |\mathcal{E}_c|) \log(|\mathcal{V}_c|))$. The mask learning component of SGExplainer has complexity $O(T|\mathcal{E}_c|)$ as it learns a mask on $\mathcal{E}_c$ for each instance. Thus, for I_t instances, the overall training complexity is as stated in the proposition. In practice, SGExplainer can achieve strong performance with only a small number of training instances, as demonstrated in Section 5.2.

PROPOSITION 2. *Let I denote the number of instances to be explained, where each instance has a computation graph $\mathcal{G}_c = (\mathcal{V}_c, \mathcal{E}_c)$, the inference complexity of SGExplainer is $O(I(|\mathcal{E}_c| + \Delta_{\text{path}}))$.*

The parameterized SGExplainer is shared across all instances and, once trained, can be applied to new instances in an inductive setting. Moreover, the number of parameters of SGExplainer is independent of the size of input graph. Its inference complexity consists of two parts: the linear-time mask learning with respect to $|\mathcal{E}_c|$ and balanced path search cost Δ_{path} . Thus, SGExplainer is well-suited for real-world applications involving numerous instances that require explanation.

Algorithm 1: SGExplainer

Input: signed graph $\mathcal{G}$, trained SGNN model $\Phi(\cdot, \cdot)$ with $\mathcal{L}$ layers, explainer Ψ , instance set $I = I_t \cup \Delta I$, maximum length l_{max} and edge budget B .

```

1 for each node pair  $(s, t) \in I_t$  do
2    $\mathcal{G}_c \leftarrow$  extract the computation graph;
3    $Y \leftarrow \Phi(\mathcal{G}_c, (s, t))$ ;
4    $\mathcal{E}_{\text{path}} \leftarrow \text{SBPSearch}(\mathcal{G}_c, s, t, 2\mathcal{L})$ ;
   /* Training phase */
5 for each epoch do
6   for each node pair  $(s, t) \in I_t$  do
7      $M \leftarrow$  compute the edge mask using Eq. (2);
8      $\mathcal{L}_{\text{pred}}, \mathcal{L}_{\text{path}} \leftarrow$  compute loss using Eq. (4), (5);
9      $\mathcal{L}_{\text{total}} \leftarrow$  compute the total loss using Eq. (3);
10    update  $\Psi$  via backpropagation;
   /* Inference phase */
11 for each node pair  $(s, t) \in \Delta I$  do
12    $M \leftarrow$  compute the edge mask using Eq. (2);
13    $\mathcal{P}(s, t) \leftarrow \text{SBPSearch}(\mathcal{G}_c, s, t, l_{\text{max}}, M, B)$ ;
14 return  $\{\mathcal{P}(s, t) \mid (s, t) \in I\}$ 

```

5 Experiments

In this section, we conduct experiments to answer the following key questions:

- **Explanation Quality and Compatibility (Q1):** Can SGExplainer provide more faithful explanations for various state-of-the-art SGNN models than existing explainers?
- **Efficiency and Training Scale (Q2):** How computationally efficient is SGExplainer and how does the explanation quality vary with the number of training instances?
- **Effect of Path-enforcing Loss and Edge Budget (Q3):** What is the impact of the path-enforcing loss on explanation quality? How does the edge budget affect the explanation faithfulness?
- **Human-Understandability (Q4):** Do the explanations generated by SGExplainer align with human intuition and domain knowledge?

5.1 Experimental Setup

Datasets and SGNNs. We adopt four publicly available signed network datasets in our experiments: BitcoinOTC (a trust network for trading bitcoin), Twitter (an opinion network on the 2016 Italian Constitutional Referendum), Slashdot (a social network collected from technology news site) and Epinions (a transaction review network sourced from a trading platform). For each dataset, we train three representative SGNN models to be explained: SGCN [8], SNEA [18], and LightSGCN [19]. All trained models achieve a link sign prediction AUC of at least 85% on all datasets. Detailed datasets statistics and training procedures are provided in Appendix B.

Explainer Baselines. We compare SGExplainer against three popular XGNN methods: GNNExplainer [30], PGExplainer [21] and PaGE-Link [35]. The adapted versions for explaining LSP are referred to as GNNExp-Link and PGExp-Link. For each dataset, we randomly sample 1,000 node pairs from the test sets for link sign prediction and explanation generation. We set the maximum path length l_{max} to 5, the edge budget B to 40, and the hyperparameter λ to 0.5. All experiments are conducted on a machine equipped with an RTX 3070 GPU. Further details on explainer training and hyperparameter settings are provided in Appendix B.3.

Evaluation Metrics. We adopt the widely used *Fidelity* [32] and *Charact* [1] metrics to evaluate explanation quality. The fidelity metrics assess the contribution of generated explanations to model predictions: *Fid-* measures factual sufficiency (lower is better), and *Fid+* measures counterfactual necessity (higher is better). The *Charact* score combines both perspectives to provide a balanced measure of explanation quality, rewarding those that are both sufficient and necessary, and is defined as:

$$\text{Charact} = \frac{w_+ + w_-}{\frac{w_+}{\text{Fid}^+} + \frac{w_-}{1 - \text{Fid}^-}} \quad (7)$$

where $w_+ = w_- = 0.5$. Detailed definition of *Fid+* and *Fid-* are provided in Appendix B.4. In summary, higher *Fid+* and *Charact* values indicate better performance, while lower *Fid-* values are preferable.

5.2 Experimental Results

Explanation Quality and Compatibility Evaluation (Q1) Table 2 shows that our proposed method, SGExplainer, consistently achieves the highest *Fid+* and *Charact* across all datasets and models, while maintaining low *Fid-* scores. This indicates that SGExplainer produces both sufficient and necessary explanations with

Table 2: Comparison of explanation methods on three representative SGNN models. Higher *Fid+* and *Charact* values indicate better performance, while lower *Fid-* values are preferable.

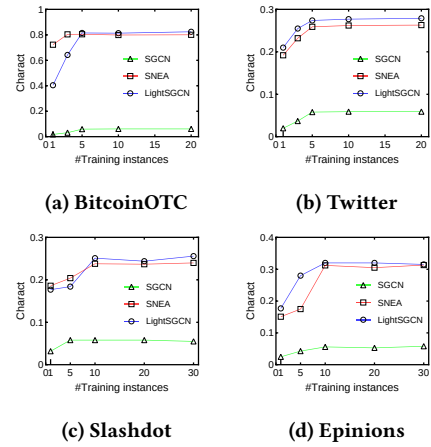
Method	Model	BitcoinOTC			Twitter			Slashdot			Epinions		
		<i>Fid+</i> ↑	<i>Fid-</i> ↓	<i>Charact</i> ↑	<i>Fid+</i> ↑	<i>Fid-</i> ↓	<i>Charact</i> ↑	<i>Fid+</i> ↑	<i>Fid-</i> ↓	<i>Charact</i> ↑	<i>Fid+</i> ↑	<i>Fid-</i> ↓	<i>Charact</i> ↑
GNNExp-Link	SGCN	0.009	0.047	0.018	0.008	0.025	0.016	0.007	0.008	0.013	0.014	0.081	0.028
	SNEA	0.003	0.041	0.006	0.050	0.065	0.095	0.087	0.073	0.159	0.183	0.083	0.305
	LightSGCN	0.012	0.042	0.024	0.060	0.075	0.113	0.095	0.078	0.172	0.165	0.081	0.280
PGExp-Link	SGCN	0.008	0.066	0.016	0.006	0.045	0.012	0.001	0.016	0.002	0.009	0.076	0.017
	SNEA	0.041	0.042	0.079	0.080	0.065	0.147	0.105	0.076	0.189	0.082	0.082	0.151
	LightSGCN	0.056	0.044	0.106	0.090	0.075	0.164	0.113	0.081	0.201	0.095	0.084	0.172
PaGE-Link	SGCN	0.011	0.044	0.021	0.012	0.040	0.024	0.021	0.029	0.042	0.006	0.039	0.013
	SNEA	0.646	0.042	0.772	0.070	0.080	0.130	0.046	0.088	0.088	0.096	0.074	0.174
	LightSGCN	0.619	0.039	0.753	0.080	0.085	0.147	0.057	0.093	0.107	0.102	0.077	0.184
SGExplainer	SGCN	0.031	0.021	0.060	0.030	0.018	0.059	0.030	0.011	0.058	0.029	0.039	0.056
	SNEA	0.686	0.025	0.805	0.150	0.060	0.259	0.136	0.063	0.238	0.187	0.050	0.312
	LightSGCN	0.698	0.021	0.815	0.160	0.058	0.274	0.145	0.060	0.251	0.192	0.048	0.320

all SGNN backbone models for the LSP task, demonstrating better effectiveness and robustness compared to existing methods. In contrast, baseline methods often exhibit imbalanced performance, which undermines their explanatory utility. Among these methods, both the path-based PaGE-Link and SGExplainer outperform subgraph-based alternatives. This supports the effectiveness of the path-enforcing learning scheme for explaining link prediction task, which encourages the explainer to focus on structural constrained paths, thereby improving its explanation performance. Moreover, explainers built on SNEA and lightSGCN backbones achieve better overall scores across all metrics compared to those based on SGCN. We attribute that to their enhanced representational capacity, which yields more structured and distinguishable latent spaces, ultimately enabling more interpretable behaviors. Overall, the consistent improvements achieved by SGExplainer highlight its superior explanation performance, making it a robust and reliable tool for interpreting LSP task.

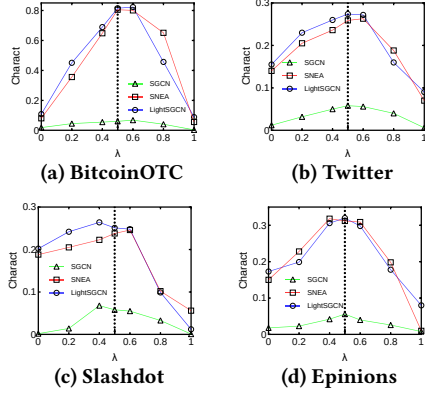
Efficiency Evaluation (Q2) Table 3 compares the running time of PaGE-Link and SGExplainer in generating path-based explanations for 1,000 predictions made by SGCN model. Since GNNExp-Link and PGExp-Link do not perform path search and instead select top- B edges as explanations, they are excluded from this comparison to ensure fairness. We observe that SGExplainer consistently

outperforms PaGE-Link in terms of runtime, with a significantly lower average per-instance runtime (ranging from 0.82 to 7.78s) compared to PaGE-Link (17.48 to 40.81s). The speedup factor varies by dataset, reaching up to approximately $21\times$ faster on BitcoinOTC and around $5\times$ on Epinions.

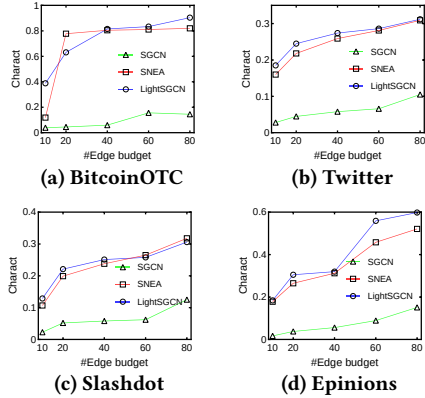
Explanation Quality vs. Training Size (Q2) Figure 4 presents the explanation quality of SGExplainer on four datasets across varying training instances. We vary the number of training instances $I_t \in \{1, 5, 10, 15, 20, 30\}$, using the remaining instances for testing. The results yield the following observations: 1) The *Charact* score improves as the number of training instances increases, indicating that SGExplainer can effectively learn and generalize shared motifs across instances, thereby enhancing explanation robustness. 2) SGExplainer achieves good performance even with limited training instances, such as 5 for BitcoinOTC and Twitter. The above two experiments for Q2 demonstrate that SGExplainer provides a more efficient and scalable solution for generating path-based explanations, making it well-suited for large-scale settings.

**Figure 4: Evaluation of SGExplainer on explanation quality vs. number of training instances.****Table 3: Runtime (including pretraining, training, and inference) comparison between PaGE-Link and SGExplainer for generating path-based explanations. $\mathcal{G}_c$ size denotes the number of nodes and edges in the computational graph.**

Method	Dataset	Avg. $\mathcal{G}_c$ size	Avg. runtime (s)
PaGE-Link	BitcoinOTC	410 / 1,971	17.48±1.54
SGExplainer			0.82±0.21
PaGE-Link	Twitter	880 / 4,300	19.37±1.62
SGExplainer			1.84±0.27
PaGE-Link	Slashdot	1,370 / 7,685	21.55±1.96
SGExplainer			3.39±0.43
PaGE-Link	Epinions	3,428 / 47,689	40.81±1.82
SGExplainer			7.78±0.55

Figure 5: Effects of hyperparameter λ on SGExplainer.

Ablation Studies on Hyperparameter λ (Q3) The training objective of SGExplainer includes one hyperparameter λ , which balances the two terms in Eq. (3). By adjusting λ , we control the trade-off between generating explanatory edges that strongly influence the model’s prediction and ensuring that these edges form meaningful balanced paths. Figure 5 shows the impact of varying λ for explaining SGNNs on four datasets. When $\lambda = 0$, the *Charact* score is relatively low, indicating that optimizing solely for the prediction loss is insufficient for generating high-quality explanations in the LSP task. As λ increases, the *Charact* score generally improves, suggesting that incorporating the balanced-path-enforcing objective enhances explanation quality. Generally, $\lambda \in [0.4, 0.6]$ achieves a good trade-off between prediction fidelity and path-enforcing learning. However, when λ approaches 1, we observe a sharp drop in performance, particularly at $\lambda = 1$. Therefore, both the prediction fidelity and balanced-path objectives must be jointly optimized, as they complement each other in producing faithful and understandable explanations for the LSP task.

Figure 6: Effects of edge budget B on SGExplainer.

Ablation Studies on Edge Budget B (Q3) Figure 6 illustrates the effects of edge budget B on the explanation quality of SGExplainer. We vary the edge budget from $[10, 20, 40, 60, 80]$ and evaluate the resulting *Charact* score. The findings are as follows: 1) The *Charact* score increases as more edges are included in the explanations, aligning with our expectation that incorporating more edges captures richer sign interactions and leads to more complete

and higher-quality explanations. 2) For the BitcoinOTC dataset, a relatively low edge budget (e.g., 20 edges for explaining SNEA) is sufficient to achieve high explanation quality, with the *Charact* score stabilizing as more edges are added. In contrast, for the larger datasets, the positive correlation between edge budget and explanation quality remains evident, and SGExplainer continues to benefit from a larger edge budget.

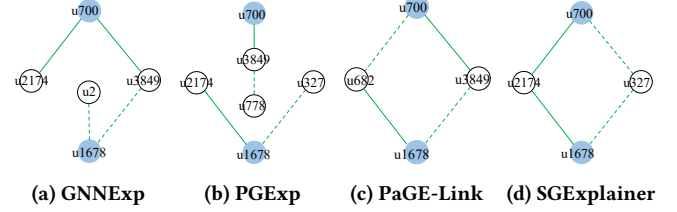


Figure 7: Explanations (green lines) for the predicted “friend” relation between blue nodes (u700 and u1678) on Slashdot. Solid/dashed lines denote ‘friend/foe’ relations.

Case Study (Q4) Figure 7 visualizes the explanations generated by baselines and SGExplainer for friend/foe prediction on social network Slashdot, using a fixed edge budget of $B = 4$ for fair comparison. We observe that the explanations in Fig. 7(a)(b) produced by GNNExp and PGExp are less interpretable, as the top-ranked important edges identified by these methods are often disconnected or loosely connected from the source and target users. In contrast, both PaGE-Link and SGExplainer (Fig. 7(c)(d)) provide two paths as explanations, which more clearly illustrate the connection between the source and target users. While PaGE-Link sometimes produces the same explanations as GNNExp, despite the latter not employing any path-enforcing mechanism. This can be attributed to early training noise that weakens its path learning module. Notably, SGExplainer successfully explains the predicted friend relation through semantically meaningful and socially intuitive patterns along the generated balanced paths: “the friend of a friend is a friend” and “the enemy of an enemy is a friend”, offering superior explanations that align with human commonsense. In comparison, PaGE-Link overlooks sign interaction and includes noisy or irrelevant edges, diminishing the human-understandability of its results.

6 Conclusion

In this work, we investigate on balanced-path-based SGNN explanation for LSP, and propose SGExplainer, the first post-hoc method that advances the transparency of SGNN models. By leveraging the concept of balanced paths, it addresses critical challenges faced by existing XGNN methods, such as disconnected subgraphs and overlooked sign interactions. SGExplainer generates faithful explanations with interpretable sign connections that are well-aligned with human knowledge. Extensive experiments demonstrate its superiority over state-of-the-art methods.

Acknowledgments

This work was supported in part by UK Research and Innovation (UKRI) Grant No. EP/Y036786/1, and Horizon Europe Grant No. 101129910.

References

- [1] Kenza Amara, Rex Ying, Zitao Zhang, Zhihao Han, Yinan Shan, Ulrik Brandes, Sebastian Schemm, and Ce Zhang. 2022. Graphframex: Towards systematic evaluation of explainability methods for graph neural networks. *arXiv preprint arXiv:2206.09677* (2022).
- [2] Francesco Bonchi, Edoardo Galimberti, Aristides Gionis, Bruno Ordóñez, and Giancarlo Ruffo. 2019. Discovering polarized communities in signed networks. In *Proceedings of the 28th acm international conference on information and knowledge management*. 961–970.
- [3] Roshni Chakraborty, Ritwika Das, and Joydeep Chandra. 2023. SigGAN: Adversarial model for learning signed relationships in networks. *ACM Transactions on Knowledge Discovery from Data* 17, 1 (2023), 1–20.
- [4] Bhanu Chander, Chinju John, Lekha Warrier, and Kumaravelan Gopalakrishnan. 2025. Toward trustworthy artificial intelligence (TAI) in the context of explainability and robustness. *Comput. Surveys* 57, 6 (2025), 1–49.
- [5] Ben Chester Cheong. 2024. Transparency and accountability in AI systems: safeguarding wellbeing in the age of algorithmic decision-making. *Frontiers in Human Dynamics* 6 (2024), 1421273.
- [6] Tyler Derr. 2020. Network analysis with negative links. In *Proceedings of the 13th international conference on web search and data mining*. 917–918.
- [7] Tyler Derr, Charu Aggarwal, and Jiliang Tang. 2018. Signed network modeling based on structural balance theory. In *Proceedings of the 27th ACM international conference on information and knowledge management*. 557–566.
- [8] Tyler Derr, Yao Ma, and Jiliang Tang. 2018. Signed graph convolutional networks. In *2018 IEEE International Conference on Data Mining (ICDM)*. IEEE, 929–934.
- [9] Rudresh Dwivedi, Devam Dave, Het Naik, Smriti Singhal, Rana Omer, Pankesh Patel, Bin Qian, Zhenyu Wen, Tejal Shah, Graham Morgan, et al. 2023. Explainable AI (XAI): Core ideas, techniques, and solutions. *Comput. Surveys* 55, 9 (2023), 1–33.
- [10] Ernesto Estrada. 2019. Rethinking structural balance in signed social networks. *Discrete Applied Mathematics* 268 (2019), 70–90.
- [11] Junjie Huang, Huawei Shen, Liang Hou, and Xueqi Cheng. 2021. SDGNN: Learning node representation for signed directed networks. In *Proceedings of the AAAI conference on artificial intelligence*, Vol. 35. 196–203.
- [12] Shuyi Jin, Mengji Zhang, Meijie Wang, and Lun Yu. 2024. A deep graph model for the signed interaction prediction in biological network. *arXiv preprint arXiv:2407.07357* (2024).
- [13] Min-Jeong Kim, Yeon-Chang Lee, and Sang-Wook Kim. 2023. TrustSGCN: learning trustworthiness on edge signs for effective signed graph convolutional networks. In *Proceedings of the 46th International ACM SIGIR Conference on Research and Development in Information Retrieval*. 2451–2455.
- [14] Yeon-Chang Lee, Nayoun Seo, Kyungsik Han, and Sang-Wook Kim. 2020. Asine: Adversarial signed network embedding. In *Proceedings of the 43rd international acm sigir conference on research and development in information retrieval*. 609–618.
- [15] Bruno Lepri, Nuria Oliver, Emmanuel Letouze, Alex Pentland, and Patrick Vinck. 2018. Fair, transparent, and accountable algorithmic decision-making processes: The premise, the proposed solutions, and the open challenges. *Philosophy & Technology* 31, 4 (2018), 611–627.
- [16] Lu Li, Jiale Liu, Xingyu Ji, Maojun Wang, and Zeyu Zhang. 2024. Se-sgformer: A self-explainable signed graph transformer for link sign prediction. *arXiv preprint arXiv:2408.08754* (2024).
- [17] Xuhong Li, Mengnan Du, Jiamin Chen, Yekun Chai, Himabindu Lakkaraju, and Haoyi Xiong. 2023. M4: A Unified XAI Benchmark for Faithfulness Evaluation of Feature Attribution Methods across Metrics, Modalities and Models. *Advances in Neural Information Processing Systems* 36 (2023), 1630–1643.
- [18] Yu Li, Yuan Tian, Jiawei Zhang, and Yi Chang. 2020. Learning signed network embedding via graph attention. In *Proceedings of the AAAI conference on artificial intelligence*, Vol. 34. 4772–4779.
- [19] Haoxin Liu. 2022. Lightsgcn: Powering signed graph convolution network for link sign prediction with simplified architecture design. In *Proceedings of the 45th International ACM SIGIR Conference on Research and Development in Information Retrieval*. 2680–2685.
- [20] Haoxin Liu, Ziwei Zhang, Peng Cui, Yafeng Zhang, Qiang Cui, Jiashuo Liu, and Wenwu Zhu. 2021. Signed graph neural network with latent groups. In *Proceedings of the 27th ACM SIGKDD conference on knowledge discovery & data mining*. 1066–1075.
- [21] Dongsheng Luo, Wei Cheng, Dongkuan Xu, Wenchao Yu, Bo Zong, Haifeng Chen, and Xiang Zhang. 2020. Parameterized explainer for graph neural network. *Advances in neural information processing systems* 33 (2020), 19620–19631.
- [22] Haitao Mao, Juanhui Li, Harry Shomer, Bingheng Li, Wenqi Fan, Yao Ma, Tong Zhao, Neil Shah, and Jiliang Tang. 2024. Revisiting Link Prediction: a data perspective. In *The Twelfth International Conference on Learning Representations*. <https://openreview.net/forum?id=8Ur2xmuw7w>
- [23] Joao Marques-Silva and Alexey Ignatiev. 2022. Delivering trustworthy AI through formal XAI. In *Proceedings of the AAAI Conference on Artificial Intelligence*, Vol. 36. 12342–12350.
- [24] Marco Minici, Federico Cinus, Francesco Bonchi, and Giuseppe Manco. 2024. Link Polarity Prediction from Sparse and Noisy Labels via Multiscale Social Balance. In *Proceedings of the 33rd ACM International Conference on Information and Knowledge Management*. 1689–1699.
- [25] Zahra Sadeghi, Roohallah Alizadehsani, Mehmet Akif Cifci, Samina Kausar, Rizwan Rehman, Priyakshi Mahanta, Pranjal Kumar Bora, Ammar Almasri, Rami S Alkhalwaldeh, Sadiq Hussain, et al. 2024. A review of Explainable Artificial Intelligence in healthcare. *Computers and Electrical Engineering* 118 (2024), 109370.
- [26] Changwon Seo, Kyeong-Joong Jeong, Sungsu Lim, and Won-Yong Shin. 2022. SiReN: Sign-aware recommendation using graph neural networks. *IEEE Transactions on Neural Networks and Learning Systems* 35, 4 (2022), 4729–4743.
- [27] Lin Shu, Erxin Du, Yaomin Chang, Chuan Chen, Zibin Zheng, Xingxing Xing, and Shaofeng Shen. 2021. Sgcl: Contrastive representation learning for signed graphs. In *Proceedings of the 30th ACM international conference on information & knowledge management*. 1671–1680.
- [28] Jiliang Tang, Charu Aggarwal, and Huan Liu. 2016. Recommendations in signed social networks. In *Proceedings of the 25th international conference on world wide web*. 31–40.
- [29] Wei Jie Yeo, Wihan Van Der Heever, Rui Mao, Erik Cambria, Ranjan Satapathy, and Gianmarco Mengaldo. 2025. A comprehensive review on financial explainable AI. *Artificial Intelligence Review* 58, 6 (2025), 1–49.
- [30] Zitao Ying, Dylan Bourgeois, Jiaxuan You, Marinka Zitnik, and Jure Leskovec. 2019. Gnnexplainer: Generating explanations for graph neural networks. *Advances in neural information processing systems* 32 (2019).
- [31] Hao Yuan, Jiliang Tang, Xia Hu, and Shuiwang Ji. 2020. Xggn: Towards model-level explanations of graph neural networks. In *Proceedings of the 26th ACM SIGKDD international conference on knowledge discovery & data mining*. 430–438.
- [32] Hao Yuan, Haiyang Yu, Shurui Gui, and Shuiwang Ji. 2022. Explainability in graph neural networks: A taxonomic survey. *IEEE transactions on pattern analysis and machine intelligence* 45, 5 (2022), 5782–5799.
- [33] Hao Yuan, Haiyang Yu, Jie Wang, Kang Li, and Shuiwang Ji. 2021. On explainability of graph neural networks via subgraph explorations. In *International conference on machine learning*. PMLR, 12241–12252.
- [34] Shichang Zhang, Yozen Liu, Neil Shah, and Yizhou Sun. 2022. Gstarx: Explaining graph neural networks with structure-aware cooperative games. *Advances in neural information processing systems* 35 (2022), 19810–19823.
- [35] Shichang Zhang, Jiani Zhang, Xiang Song, Soji Adeshina, Da Zheng, Christos Faloutsos, and Yizhou Sun. 2023. PaGe-Link: Path-based graph neural network explanation for heterogeneous link prediction. In *Proceedings of the ACM Web Conference 2023*. 3784–3793.
- [36] Zeyu Zhang, Jiamou Liu, Xianda Zheng, Yifei Wang, Pengqian Han, Yupan Wang, Kaiqi Zhao, and Zijian Zhang. 2023. Rsgnn: A model-agnostic approach for enhancing the robustness of signed graph neural networks. In *Proceedings of the ACM web conference 2023*. 60–70.
- [37] Zeyu Zhang, Peiyao Zhao, Xin Li, Jiamou Liu, Xinrui Zhang, Junjie Huang, and Xiaofeng Zhu. 2024. Signed Graph Representation Learning: A Survey. *arXiv preprint arXiv:2402.15980* (2024).
- [38] Peiyao Zhao, Xin Li, Zeyu Zhang, Mingzhong Wang, Xueying Zhu, and Lejian Liao. 2025. Robust Deep Signed Graph Clustering via Weak Balance Theory. In *Proceedings of the ACM on Web Conference 2025*. 3819–3830.

A Algorithms

A.1 Notations

Table 4 summarizes the necessary notations used in this paper.

A.2 SGExplainer

Algorithm 1 describes the full process of SGExplainer. The SGExplainer algorithm generates human-understandable balanced-path explanations for link sign predictions made by SGNN model. Given a signed graph $\mathcal{G}$, a trained SGNN model Φ , and a parameterized explainer Ψ , the algorithm extracts a computation graph $\mathcal{G}_c$ for each node pair (s, t) , computes the SGNN prediction Y , and identifies balanced paths $\mathcal{E}_{path}$ in $\mathcal{G}_c$. In the training phase, Ψ learns to assign importance weights to graph edges using a set of training instances I_t over T epochs. For each instance, the algorithm computes an edge mask M and optimizes a loss function composed of three terms: prediction loss $\mathcal{L}_{pred}$ to ensure faithfulness to the SGNN's decision, path-enforcing loss $\mathcal{L}_{path}$ to encourage explanations structured around balanced paths, and total loss $\mathcal{L}_{total}$ for overall optimization. During inference, Ψ is applied to all instances in I , computing edge masks and conduct shortest balanced path search based on edge importance scores. The algorithm then returns the set of balanced paths $\mathcal{P}(s, t)$ as explanation for each instance.

A.3 Shortest balanced path search SBPSearch

Algorithm 2: SBPSearch

Input: Signed graph $\mathcal{G}$, source s , target t , target sign $\sigma \in \{+1, -1\}$, max path length $l_{max} = 2\mathcal{L}$ (default), edge mask M (optional), edge budget B (optional)
Output: Set $\mathcal{P}$ of balanced paths from s to t with sign σ

```

1 foreach edge  $(u, v) \in \mathcal{G}$  do
2   build edge sign function  $\text{sign}[(u, v)] \leftarrow \text{sign of } (u, v)$ 
3 if  $M$  is provided then
4   foreach edge  $(u, v) \in \mathcal{G}$  do
5     initialize weight function  $w[(u, v)] \leftarrow M_{(u, v)}$ 
6 else
7   foreach edge  $(u, v) \in \mathcal{G}$  do
8      $w[(u, v)] \leftarrow 1$ 
9 if  $B$  is provided then
10   $k \leftarrow \max\{1, \lfloor B/l_{max} \rfloor\}$ 
11 else
12   $k \leftarrow +\infty$ 
13  $\text{paths} \leftarrow \text{Dijkstra}(\mathcal{G}, s, t, w)$ 
14 foreach  $p$  in  $\text{paths}$  do
15   $\sigma_p \leftarrow \prod_{(u, v) \in p} \text{sign}[(u, v)]$ 
16  if  $\sigma_p = \sigma$  and  $|p| \leq l_{max}$  then
17    Append  $p$  to  $\mathcal{P}$ ;
18    if  $|\mathcal{P}| \geq k$  then
19      break
20 return  $\mathcal{P}(s, t)$ 

```

Algorithm 2 describes the shortest balanced path search SBPSearch algorithm. It first initializes edge weights and signs, then determines

the maximum number of paths allowed based on an edge budget and maximum path length. Using a bidirectional Dijkstra search, it finds shortest paths and filters them to keep only those with the desired overall sign and within the length limit. The algorithm returns up to the specified number of balanced, length-constrained shortest paths efficiently. The time complexity of SBPSearch primarily depends on two stages. The first stage runs a bidirectional Dijkstra algorithm to find shortest paths between the source and target nodes, which operates in $O((|\mathcal{V}| + |\mathcal{E}|) \log(|\mathcal{V}|))$ for signed graph $\mathcal{G} = (\mathcal{V}, \mathcal{E})$. The second stage involves iterating over the set of shortest paths returned by the first stage to filter them based on path sign and length constraints. Assuming that the number of shortest paths is polynomially bounded, this filtering step runs in $O(kl)$, where k is the number of shortest paths and l is the path length. Since both k and l are typically much smaller than the size of the graph, the filtering adds only a modest overhead. Therefore, the overall time complexity remains close to $O((|\mathcal{V}| + |\mathcal{E}|) \log(|\mathcal{V}|))$ in practice.

A.4 Complexity Analysis of XGNN methods

Table 5 summarizes the time complexity of SGExplainer and other representative methods to explain predictions for I instances. Let T be the mask learning epochs and I be the number of instances to be explained, each instance with computation graph $\mathcal{G}_c = (\mathcal{V}_c, \mathcal{E}_c)$ of a full graph $\mathcal{G} = (\mathcal{V}, \mathcal{E})$. The time complexity of the balanced path search is denoted by $\Delta_{path} = (|\mathcal{V}_c| + |\mathcal{E}_c|) \log(|\mathcal{V}_c|)$. Due to PGExplainer [21] and SGExplainer have a training stage and an inference stage (separated by / in the table), let I_t be the number of instances for training explainers.

GNNEExplainer [30] has complexity $O(IT|\mathcal{E}_c|)$ as it learns a mask on $\mathcal{E}_c$ for each instance. For PGExplainer [21], the inference stage is linear in $|\mathcal{E}_c|$ for each instance, so the inference complexity is $O(I|\mathcal{E}_c|)$. Its training stage learns a mask generator for generating edge mask for each instances, and cover edges in the computational graphs of I_t training instances, thus the complexity is $O(I_t T |\mathcal{E}_c|)$. SubgraphX [33] has a much higher time complexity $O(I|\mathcal{V}_c| \hat{D}^{2B_{node}-2})$ as analyzed in [35], where $\hat{D} = \max_{v \in \mathcal{V}} D_v$ denotes the maximum degree and B_{node} denotes the node budget for explanations. PaGE-Link [35] learns an edge mask for each instance, resulting in the time complexity of $O(IT(\Delta_{path} + |\mathcal{E}_c|))$, in which the complexity of mask learning using Dijkstra's algorithm at each epoch is $\Delta_{path} + |\mathcal{E}_c|$. Here, we ignore the linear k-core pruning stage in PaGE-Link and focus on the mask learning stage. For our SGExplainer, its training complexity consists of two parts: the complexity Δ_{path} of balanced path search step and $T|\mathcal{E}_c|$ for mask learning step. Thus, the total complexity with I_t training instances is $O(I_t(\Delta_{path} + T|\mathcal{E}_c|))$. Its inference complexity consists of two parts: the linear-time mask learning with respect to $|\mathcal{E}_c|$ and balanced path search cost Δ_{path} . Thus, the overall inference complexity is $O(I(|\mathcal{E}_c| + \Delta_{path}))$. Therefore, SGExplainer has better training time complexity than PaGE-Link by decoupling path search and mask learning. Furthermore, once SGExplainer is trained on a small set of I_t instances, it can explain new instances inductively without retraining, unlike Page-Link. Both PaGE-Link and SGExplainer often converge faster, i.e., have a smaller T , due to the smaller space of candidate explanations from paths.

Table 4: Notations

Notation	Meaning/Definition
$\mathcal{G} = (\mathcal{V}, \mathcal{E}^+, \mathcal{E}^-)$	a signed graph with node set $\mathcal{V}$, positive edge set $\mathcal{E}^+$ and negative edge set $\mathcal{E}^-$
(s, t)	the node pair including a source node and target node
$\Phi(\cdot, \cdot), \Phi_e, \Phi_d$	the SGNN model to be explained, the inside encoder and decoder
z_s, z_t	node representations of node s and t generated by Φ_e
$Y = \Phi(\mathcal{G}, (s, t))$	link sign prediction Y of SGNN model $\Phi(\cdot, \cdot)$ for node pair (s, t)
$\mathcal{L}$	the number of layers of SGNN encoder
$\mathcal{G}_c = (\mathcal{V}_c, \mathcal{E}_c^+, \mathcal{E}_c^-)$	the computation graph, i.e., $\mathcal{L}$ -hop ego-graph of node pair (s, t)
$\text{dist}(u, v)$	the length of shortest path between node u and v
$l_{\max}$	the maximum length of explanatory balanced paths
B	the maximum number of edges included in the explanation
$\Psi(\cdot)$	the mask generator model
M	the generated edge mask of mask generator Ψ
$\mathcal{G}_m = (\mathcal{V}_c, \sigma(M) \odot \mathcal{E}_c)$	the masked computation graph
λ	the weight of the loss
$I = I_t \cup \Delta I$	the set of instances to be explained, I_t denotes training instances
$\mathcal{P}$	the generated balanced path set as explanation for each instance $(s, t) \in I$
Δ_{path}	the path search time complexity

Table 5: Time complexity of explainer methods for explaining I instances.

Method	Time Complexity
GNNExplainer	$O(IT \mathcal{E}_c)$
PGExplainer	$O(I_t T \mathcal{E}_c) / O(I \mathcal{E}_c)$
SubgraphX	$O(I \mathcal{V}_c \hat{D}^{2B_{\text{node}}-2})$
PaGE-Link	$O(IT(\Delta_{\text{path}} + \mathcal{E}_c))$
SGExplainer	$O(I_t(\Delta_{\text{path}} + T \mathcal{E}_c)) / O(I(\mathcal{E}_c + \Delta_{\text{path}}))$

B Experimental Settings

We implement the explainers and SGNN models using PyTorch and the DGL library. All experiments are conducted on a machine equipped with an RTX 3070 GPU. For each dataset, we first train the SGNNs and then use the explainers to interpret the link sign predictions made by the well-trained SGNNs.

B.1 Dataset Statistic

We adopt four widely used signed network datasets¹ in our experiments: BitcoinOTC, Twitter, Slashdot and Epinions. Dataset statistics are summarized in Table 6. #train and #test are the number of edges in the training and testing sets. BitcoinOTC is a signed trust network for trading bitcoin on the Bitcoin-OTC platform, where users rate others from [-10, -1] (distrust) and [1, 10] (trust) to build reputation and prevent fraud. We simplify ratings to -1 or +1 based on the given range. Twitter is a signed social network constructed from interactions among Twitter users during the 2016 Italian Constitutional Referendum, where an edge is labeled +1 if two users share the same stance on the referendum and -1 otherwise. Slashdot is a signed social network collected from technology news site Slashdot, where users mark others as friends (+1) or foes (-1) to influence content visibility. Epinions is a signed transaction

review network sourced from a trading platform Epinions where users assign trust (+1) or distrust (-1) based on transaction outcomes. Since the datasets do not include node features, we initialize one-hot features for BitcoinOTC, while for the other datasets, we generate random node features for each node.

B.2 Details of SGNNs

Our experiments involve three state-of-the-art SGNN models to be explained: SGCN [8], SNEA [18], and LightSGCN [19]. All trained SGNN models achieve a link sign prediction AUC of at least 85% on all datasets. For the implementation of SGNN models, we adopt the source codes^{2 3 4} provided by their original paper [8, 18, 19] and re-implement these models into an encoder-decoder framework to unify training and evaluation. We follow the hyperparameter settings recommended in the original papers for each model. All models are configured with two SGNN layers and a final embedding dimension of 32. Following these works, we randomly select 80% of the edges as the training set and use the remaining 20% as the testing set, all models are trained and evaluated on the same split.

B.3 Details of Explainers

We compare SGExplainer against three popular GNN explainers: GNNExplainer [30], PGExplainer [21] and PaGE-Link [35]. For the implementation of baselines, we adopt the source code reported by their original paper and use their pre-defined objectives to train the model. PaGE-Link aims to provide path-based explanations for heterogeneous link prediction, and we adapt it to explain the LSP task. Since GNNExplainer and PGExplainer are not directly applicable to link prediction, we follow the adaptation process in [35], extending them to select the top B weighted edges from their learned masks to construct edge-induced subgraph explanations.

²<https://github.com/benedekrozemberczki/SGCN>

³<https://github.com/liyu1990/snea>

⁴<https://github.com/akaranmoe/LightSGCN>

¹<https://github.com/NDS-VU/signed-network-datasets>

Table 6: The statistics of datasets

Dataset	#nodes	#positive edges	#negative edges	#train	#test
BitcoinOTC	5,881	32,029	3,563	28,473	7,119
Twitter	10,885	238,612	12,794	201,125	50,281
Slashdot	82,141	380,933	119,548	400,384	100,097
Epinions	131,580	589,888	121,322	568,968	142,242

Table 7: The hyperparameters of SGExplainer

	BitcoinOTC	Twitter	Slashdot	Epinions
number of layers	2	2	2	2
embedding dim	64	64	64	64
learning rate	0.001	0.003	0.005	0.005
# epochs	50	50	20	20
optimizer	Adam	Adam	Adam	Adam
k-hop $\mathcal{G}_c$	2	2	2	2
λ	0.5	0.5	0.5	0.5
l_{max}	5	5	5	5
B	40	40	40	40

These adapted versions are referred to as GNNExp-Link and PGExp-Link. To ensure a fair comparison, we randomly sample 1,000 node pairs from the testing set to be predicted and explained. The sampled node pairs are identical for SGNN models and explainers. Since longer paths are less meaningful and harder to interpret, we restrict the maximum length l_{max} of generated path to 5, and the edge budget B to 40. The hyperparameter λ is set to 0.5. The details of the hyperparameters of SGExplainer are shown in Table 7.

B.4 Evaluation Metrics

As ground truth explanations for LSP tasks are unavailable, we adopt the widely used fidelity [32] and charact [1] metrics to evaluate the quality of the generated explanations.

$$Fid_+ = \frac{1}{N} \sum_{i=1}^N |\Phi(\mathcal{G}_c)_{\hat{y}_i} - \Phi(\mathcal{G}_{c \setminus p})_{\hat{y}_i}| \quad (8)$$

$$Fid_- = \frac{1}{N} \sum_{i=1}^N |\Phi(\mathcal{G}_c)_{\hat{y}_i} - \Phi(\mathcal{G}_p)_{\hat{y}_i}| \quad (9)$$

$$Charact = \frac{w_+ + w_-}{\frac{w_+}{Fid_+} + \frac{w_-}{1 - Fid_-}} \quad (10)$$

where N is the number of instances, $\mathcal{G}_p$ is the subgraph induced by explanatory paths, $\mathcal{G}_{c \setminus p}$ is the computational graph excluding explanatory paths, $\hat{y}_i = \operatorname{argmax} \Phi(\mathcal{G}_c)$, and $w_+ = w_- = 0.5$. The fidelity metrics measure the contribution of the produced explanations to the initial prediction: *Fid-* evaluates factual sufficiency by measuring how well the explanation alone reproduces the original prediction, while *Fid+* evaluates counterfactual necessity by measuring the effect of removing the explanations from the original graph. The *Charact* score combines both perspectives to provide a balanced measure of explanation quality, rewarding those that are both sufficient and necessary.